

开发组加分题

写在最前

加分题的本意，是弥补短暂面试可能造成的认知不足，避免我们因信息有限而错过真正适合开发组的同学，因而我们希望不仅仅通过面试的个人回应评判个人，我们也希望能通过一些小测试来综合评价个人。

但正如上，我们的本意是为了更加均衡的了解各位大一新生，倘若各位有其余可以让我们对大家更好了解的方式，请不要顾虑，尽情的向我们展示，我们欢迎多元的自我展现。

我曾在最初希望设计一些和开发紧密相关的加分题，或直接使用SWE Bench，但我认为，这背离了我的初衷：我希望和一些对开发，对前沿科技感兴趣的同学为伍。我们不是初创公司，我们不要求你已经成为成熟的技术型人才，也不会把既有技术积累作为唯一标准，相反，我们需要的是对开发抱有热情，对前沿领域怀抱浓厚兴趣的同学，这才是我们的初衷。

本次加分题可以分为三大类：**科普情景类、实操实践类、心智模型类**。这三类题分别观察你**是否对前沿技术怀有好奇，是否愿意动手实践并驾驭工具，以及是否能够在陌生规则中建立自己的心智模型**三个维度。我们不要求你最终呈现的答卷面面俱到，正如前述，我们是希望通过加分题对各位有一个更好的了解，你只需要展示自己优秀的一面，我们自会发现，加油！

我其实非常希望大家在回答加分题时，展现出类似下面这样的思考方式。

我们更看重的并不是**你现在掌握了多少技术**，而是你是否愿意从一个问题出发，继续追问它还能怎样被理解，能否与其他领域建立联系，又能否主动质疑自己最初得到的答案。**能力与知识都可以在很长的人生中逐渐积累，不必急于在此刻证明自己。**

例如，当我们讨论**“为什么使用 LLM 编写代码时仍然需要明确约束”**，最直接的解释可能是：**LLM 的输出带有概率性，生成结果并不总是稳定。**

但思考不必停在这里。

你也可以尝试从**信息论**的角度理解：编程是在人的意图与 CPU 的行为之间建立一条语义信道。自然语言需求通常是欠定的，同一句话可能对应许多行为不同、却都看似合理的程序。如果用户没有提供足够的信息，那么模型便不得不借助训练形成的先验，对缺失的部分进行猜测。

有人可能会继续追问：**提示词和模型训练中不是已经包含了大量信息吗？**

的确如此。简短的提示词能够生成很长的代码，正是因为模型参数、软件库、项目上下文和既有编程模式承担了大量共享信息。但如果你的目标依赖某种模型训练、上下文和外部工具中都不**存在的知识**——暂且不讨论它能否由已有能力闭包得到——那么缺少的信息最终仍然需要由人、实验、检索或新的训练过程补入。此时所需的描述、搜索与验证成本可能很难预先估计，甚至可能接近重新解决问题本身。

你还可以从**偏运算或程序特化**的角度理解：LLM 写代码需要约束，不只是因为它具有随机性，还因为自然语言往往没有消除程序的全部语义自由度。类型、接口、测试、示例与不变量，都在把原本需要模型猜测的选择，提前固定为已知条件。

当然，我们并不要求大家了解信息论、偏运算或任何特定术语。这个例子只是想说明：面对一道题目时，你可以尝试**越过最直接的解释，从不同角度重新描述问题，寻找它与其他知识之间的联系，并继续提出新的问题**。即使最后的推论并不完全正确，只要能够清楚展示你的思考路径、查证过程与修正过程，它仍然是一份很有价值的回答。

作答说明：以下题目不要求全部完成，任选一道即可；你也可以完全不做这些题目，转而提交自己的项目、文章、实验记录或其他能够展示你的材料。我们更关心你的探索过程，而不是最终答案是否完整、是否完全正确。完成题目的数量不直接决定分数。

另注：加分题提交时，我们接受GitHub仓库链接或以附件形式的压缩包文件，请发送至邮箱：isseaoccupia@gmail.com，并表明个人信息，十分感谢！！



科普情景类

大语言模型漫谈

相比各位一定看到过这样的科普：

LLM就是根据已有文本，反复预测下一个token的机器

然而，无论商业模型还是开源模型，实际使用时似乎都不像在单纯“接话茬”，反而更像在理解要求并回答问题。请通过资料检索和自己的推理，解释这两种描述为什么并不矛盾，并建立你对聊天式大语言模型工作方式的理解。

以下是一些姊妹问题，你也可以尝试回答：

- 既然统计已有材料，并总结规律就可以让根据输入产生的输出精准，那么我们用圆周率作为训练，训练一个可以吐数字的机器，能不能帮我们解决计算圆周率的问题？
- 看起来，LLM生成的文本和人类生活所频繁接触的语料完全不一样，没有人说话会十分关切，或者这样头头是道，即使在人类文献中，叙述风格类似大语言模型的也寥寥无几，可为什么LLM生成的文本风格就是那样的不同？
- 能不能让LLM输出的语料来自己优化自己？

会倒着跑的程序

在CPU内部，存在一个叫做程序计数器（Program Counter，PC）的小玩意，PC 保存当前或下一条将要执行的指令地址。一般流程下，这一过程通常都是正向的，程序被加载进系统的时候，指令也是顺序排布，这一切看起来尽然就绪，吗？

注：PC是抽象称呼，在x86平台上，他被称为IP（Instruction Pointer），指令指针，指向需要运行的指令。

事实上，你完全可以让指令不顺着正常的递增地址正向排布，完全逆向的排布也是可以的。在DOS时代，这通常被称作逆指令流混淆/加密。

首先，我们请看下面一段汇编

```
mov ax, 1234h    ; AX = 1234h
inc ax           ; AX = 1235h
nop              ; 什么也不做
```

如果只让指令倒着放置，让程序逆着跑，如下

```
; 1000h
block_3:
    nop
    jmp finished

; 1010h
block_2:
    inc ax
    jmp block_3

; 1020h
entry:
    mov ax, 1234h
    jmp block_2

finished:
```

如果只看被执行的程序的排布，那确实是倒着的。有效代码块按高地址到低地址执行，但每个基本块内部仍然按照正常方向取指，并通过显式jmp连接，因此控制关系很容易被识别。这没有达到复杂的混淆，而且这种跳转在当代调试器甚至是过去的程序员手上也是非常简单的。

一种简化的基于中断器的逆指令流实现方法如下

```
; 1000h
block_3:
    nop

; 1010h
block_2:
    inc ax

; 1020h
entry:
    mov ax, 1234h
```

然后，我们可以另外准备一个流程表，例如

```
next_ip = [1010h, 1000h, finished]
```

step是一个我们需要维护的指针，用来告诉我们该执行那一段。我们可以讨论两个实现方法：

INT 1 & #DB

INT 1是CPU内部的一个中断器。程序开始之前把TF置为 1，CPU 就会在每执行完一条指令后产生调试异常，也就是 #DB，在 DOS 资料中通常称为进入INT 1处理程序

简单来说，每执行一条指令，它都会执行异常调试程序，我们可以另外通过实模式下的特性，把这个调试程序维护成我们的段地址维护器，从而实现流程自由控制。这意味着，我们其实把上述指令任意分布在内存中，只需要维护好指令流程表，如果有人尝试调试，他就会发现整个流程被拆得七零八碎，大大增大破解的难度。

INT 3

与INT 1 有差别，INT 3是一条你需要显式执行的调试指令，在执行完程序之后，你必须显式地执行int 3，不过剩余的流程差不多，维护一个流程控制器，改变运行顺序。

阅读文段，然后你需要尝试回答以下问题：

实模式指的是物理地址直接暴露给程序的CPU工作方式，1978年，8086处理器引入实模式。与实模式对应的是保护模式，你可以理解成每个程序有独立的块，块之间一般不会相互访问，这极大地减少了很多错误。

例如，在实模式下，如果存在悬空指针，或内存寻址越界，可能会导致加载在这个内存上的程序损坏或崩溃。不过实模式下也有很多有趣的故事，例如程序Novell Netware可以在被DOS加载之后直接杀死加载它的DOS系统，反客为主了属于是。

以下是问题：

- 逆指令流是DOS时代频繁使用的加密方法，为什么这种加密方法现如今逐渐退居二线？
- 既然不能访问真实的物理地址，为什么很多保留底层接口的语言还会存有“指针”这个概念？
- 现代程序加密方法有哪些，你能否尝试阐述他们是否受实模式地影响？



心智模型类

with漫谈

打开的门从未关上，泄露总是发生，于是Rossum和Coghlan发明了with。

python关键词中，with语法始终是一个非常重要的概念。他看起来不像一个成员方法，使用方法写的也很奇怪。初学者往往会在操纵文件IO部分学到它，但是教程往往一笔带过，不甚了了，暧昧不清。请你根据python文档，语言细节，IO标准库以及PEP文档，阅读后阐述你自己对这个关键词的见解。

你可能会用到：

- [PEP 343](#)
- [with statement](#)
- [pathlib](#)

张量的形状、维度与索引

孙悟空的七十二变，是哪七十二？

torch是一个划时代的机器学习库，他真正让机器学习走进千家万户。pytorch是torch的python封装，他为我们提供了一个叫做tensor的数据类型。类型之上，他也为我们提供了操纵tensor形状的方法，但是，这些方法本身涉及改变tensor的形状，看起来总是迷迷糊糊。请自行参阅官方文档，构建属于你的心智模型，你可以借助AI，但是一定要对这些成员方法对tensor做了什么操作有深刻的了解。

这一部分，我们建议你可以实际上手操作，实验和观察会带给你与LLM交谈完全不同的体验。

你可能会用到：

- [tensor tutorial](#)
- [torch.cat](#)
- [torch.narrow](#)
- [torch.permute](#)
- [torch.squeeze](#)
- [torch.unsqueeze](#)
- [torch.reshape](#)

指针偏移与容器

有了类型，我们才能更好的使用数据；有了大小，我们才能更好的储存数据。

如何衡量一个结构体内部数据偏移大小？请看下面的宏，你可以自己实验，尝试或转化成更加易读的形式，直到你可以理解它的原理

```

#if defined _MSC_VER && !defined _CRT_USE_BUILTIN_OFFSETOF
    #ifdef __cplusplus
        #define offsetof(s,m) (((::size_t)&reinterpret_cast<char const volatile>((((s*)0)->m)))
    #else
        #define offsetof(s,m) ((size_t)&(((s*)0)->m))
    #endif
#else
    #define offsetof(s,m) __builtin_offsetof(s,m)
#endif

```

由于这是一个兼容C和C++的宏，我们建议在这道题目完成前，先学习的C++基础知识，并简单对编译器，头文件，预处理等有一个基本的了解。

也可以尝试下面这个：

```

#define container_of(ptr, type, member) ({
    void *__mptr = (void *)(ptr);
    static_assert(__same_type(*(ptr), ((type *)0)->member) ||
        __same_type(*(ptr), void),
        "pointer type mismatch in container_of()");

```



实操实践类

本部分不是一个独立的测试，你可以理解为另一个维度的评测。学会使用GitHub并尝试使用GitHub仓库管理你的加分题提交材料就意味着你已经完成了本部分。我们鼓励你使用发送GitHub仓库链接的方式提交你的加分题，而并不是使用压缩包。

在选择性完成上述加分题中你喜欢的部分后（其实你也可以不做上面的加分题，如果你自己觉得自己拥有别的自我展示材料，我们也欢迎你直接提交你自己的展示材料），你也可以尝试以下的一些题目：

- 尝试使用huggingface上公布权重的模型文件，在本地部署。不局限于大语言模型，你可以尽情的尝试，记录下过程中你遇到的问题与你利用工具或自己寻找到的解决方法；
- 尝试理解为什么将python成为胶水语言，比如，你可以尝试拥有python接口的R，COLMAP或者blender，你可以尝试评估使用py文件可以串联起的工具的数量与可能性；

- 尝试理解为什么数据库存在如此复杂的语法与使用方法。从用途上，数据库只需要满足增删查改，甚至bash脚本就可以完成，但是现实中，这是一项系统级工程。
- 可以尝试了解混乱C语言编程大赛，并尝试拆解一个参赛作品，理解为什么它可以实现那种目的。

